

## CO2 – AT3 : Scenario Based Assessment — Theory of Computation

### Question 1

Given a DFA used for validating a specific PIN pattern bbbc, bca, c, ca, caaaaa — minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems.

#### Step 1: Understand the language

The valid PIN language is the finite set of exact strings:

$$L = \{ c, ca, caaaaa, bca, bbbc \} \text{ over alphabet } \Sigma = \{a, b, c\}$$

Any string that is not one of these five exact PINs must be rejected. To recognise a finite language exactly, we first build a DFA whose accepted paths are the five strings themselves (a "trie"/prefix-tree DFA), with every other transition sent to a single dead (trap) state.

#### Step 2: Construct the initial (un-minimized) DFA

Reading each PIN character by character from the start state  $q_0$  gives one branch per distinct prefix:

- $q_0 \xrightarrow{c} q_1^*$  (accepts "c")
- $q_1 \xrightarrow{a} q_3 \xrightarrow{a} q_4 \xrightarrow{a} q_5 \xrightarrow{a} q_6 \xrightarrow{a} q_7^*$  (accepts "ca", then continues to accept "caaaaa" at  $q_7$ ;  $q_3$  also accepts "ca")
- $q_0 \xrightarrow{b} q_2$
- $q_2 \xrightarrow{c} q_8 \xrightarrow{a} q_9^*$  (accepts "bca")
- $q_2 \xrightarrow{b} q_{10} \xrightarrow{b} q_{11} \xrightarrow{c} q_{12}^*$  (accepts "bbbc")
- all other missing (state, symbol) transitions go to a single dead state  $D$ , which loops to itself on every input

(\* = accepting state). This gives 13 live states + 1 dead state = 14 states in total, each accepting state marked below with a double circle.

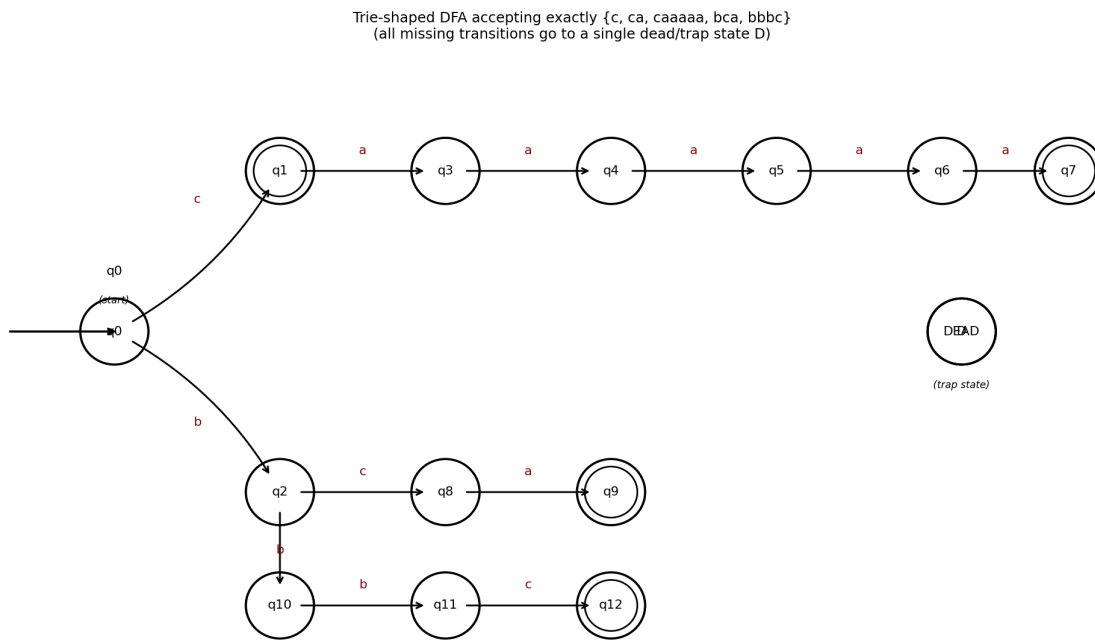


Fig. 1 — Un-minimized (trie-shaped) DFA for the PIN language

### Step 3: Minimize using equivalence-partitioning

Apply the table-filling / equivalence method:

- Partition states into  $P_0 = \{ \text{accepting states} \}$  and  $\{ \text{non-accepting states} \}$ .
- Accepting states q1, q3 have further live outgoing transitions (on 'a'), so they are distinguishable from accepting states that have no productive transitions left.
- States q7 (after "caaaaa"), q9 (after "bca") and q12 (after "bbbc") are all accepting AND, on every input symbol, they only ever go to the dead state D. Since two states are equivalent when they behave identically for every future input string,  $q_7 \equiv q_9 \equiv q_{12}$ .
- No other pair of states is equivalent — every remaining state has a different "distance" to acceptance or a different set of live transitions (e.g. q4 needs 3 more a's to accept, q5 needs 2 more — these are distinguishable by the suffix that separates them, by the Myhill–Nerode theorem).

So the only merge possible is:  $F = \{ q_7, q_9, q_{12} \}$ .

### Step 4: Minimized DFA

| Original states                                   | Count | After minimization                        | Count |
|---------------------------------------------------|-------|-------------------------------------------|-------|
| q0,q1,q2,q3,q4,q5,q6,q7,<br>q8,q9,q10,q11,q12 + D | 14    | q0,q1,q2,q3,q4,q5,q6,F,q<br>8,q10,q11 + D | 12    |

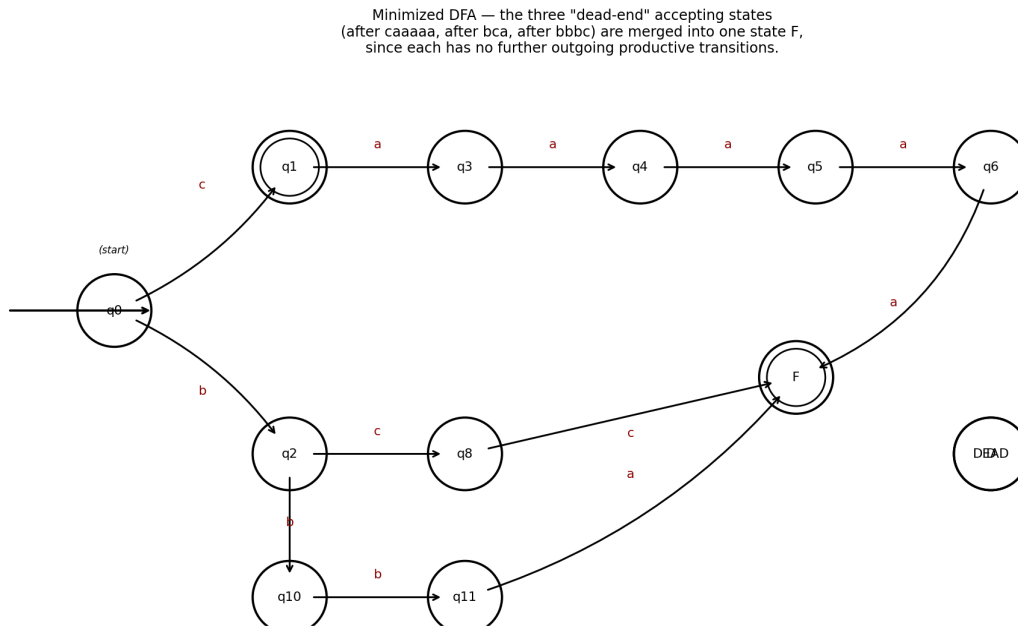


Fig. 2 — Minimized DFA (dead-end accepting states q7, q9, q12 merged into state F)

### Step 5: Why state reduction matters in embedded systems

- Lower memory footprint — the transition table shrinks from 14×3 to 12×3 entries; embedded microcontrollers often have only a few KB of flash/RAM, so every state saved is directly usable memory.
- Faster execution — fewer states means fewer comparisons per input symbol and a smaller lookup table, which fits better in cache/registers, reducing per-character processing latency — important for real-time PIN-pad validation.
- Lower power consumption — smaller tables and fewer state transitions reduce the number of memory accesses and CPU cycles, which directly reduces energy use — critical for battery-powered embedded devices such as ATMs, door locks, and POS terminals.
- Easier verification and maintenance — a minimal DFA is the unique canonical automaton for the language (by the Myhill–Nerode theorem), so it is easier to test exhaustively and to prove correct — important for security-critical PIN validation logic.

- Smaller silicon/logic footprint — if the DFA is realised directly in hardware (an FSM in an FPGA/ASIC), fewer states means fewer flip-flops and less combinational logic, reducing chip area and cost.

## Question 2

Demonstrate how a given finite automaton can be converted into a regular expression.

Note: the automaton has been redrawn below exactly as read from the given hand-drawn diagram — start state q1, states q1, q2, q3, q4, final state q4 (with a self-loop on 0,1).

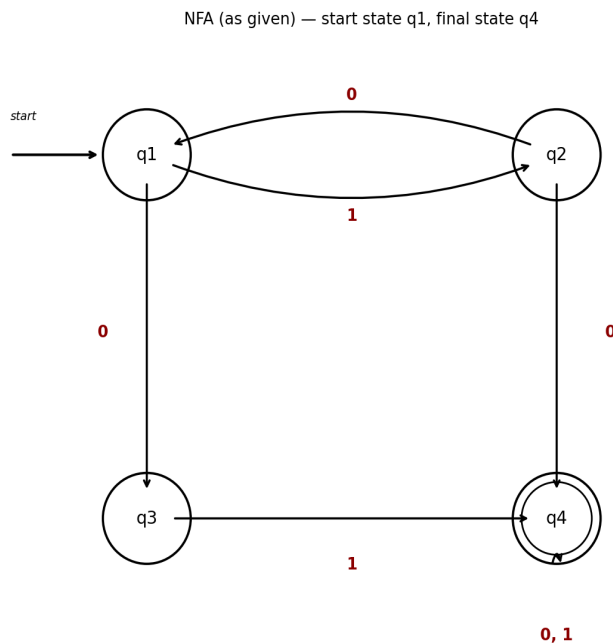


Fig. 3 — Given finite automaton

### Step 1: Read off the transitions

| State      | on 0 | on 1 |
|------------|------|------|
| q1 (start) | q2   | —    |
|            | q3   | —    |
| q2         | q4   | q1   |
| q3         | —    | q4   |
| q4 (final) | q4   | q4   |

### Step 2: Method used — Arden's Theorem (state-elimination)

Arden's Theorem: if  $X = QX + P$  and  $Q$  does not contain  $\epsilon$ , then  $X = Q^*P$  is the unique solution. We write one equation per state, expressing the set of strings that reach that state from  $q_1$ , using the incoming edges.

Let  $R_1, R_2, R_3, R_4$  denote the regular expressions for "strings that take the automaton from the start state to state  $q_1, q_2, q_3, q_4$  respectively".

$$R_1 = \epsilon + R_2 \cdot 1 \quad (\text{q1 is the start state, plus the incoming edge from q2 on 1})$$

$$R_2 = R_1 \cdot 0 \quad (\text{only incoming edge into q2 is from q1 on 0})$$

$$R_3 = R_1 \cdot 0 \quad (\text{only incoming edge into q3 is from q1 on 0})$$

$$R_4 = R_2 \cdot 0 + R_3 \cdot 1 + R_4 \cdot (0+1) \quad (\text{incoming edges from q2 on 0, q3 on 1, and its own self-loop on 0,1})$$

### Step 3: Solve $R_1$ using Arden's Theorem

$$R_1 = \epsilon + R_1 \cdot 0 \cdot 1 = \epsilon + R_1 \cdot (01)$$

$$\Rightarrow R_1 = (01)^* \quad [\text{by Arden's theorem, } X = QX + P \Rightarrow X = Q^*P, \text{ here } P = \epsilon]$$

### Step 4: Substitute back to get $R_2$ and $R_3$

$$R_2 = R_1 \cdot 0 = (01)^*0$$

$$R_3 = R_1 \cdot 0 = (01)^*0$$

### Step 5: Solve for $R_4$ (the final state) using Arden's Theorem

$$R_4 = R_4 \cdot (0+1) + (R_2 \cdot 0 + R_3 \cdot 1)$$

$$= R_4 \cdot (0+1) + ((01)^*0 \cdot 0 + (01)^*0 \cdot 1)$$

$$= R_4 \cdot (0+1) + (01)^*0 \cdot (0+1)$$

This is again of the form  $X = QX + P$  with  $Q = (0+1)$  and  $P = (01)^*0(0+1)$ , so by Arden's theorem:

$$R4 = (0+1)^* \cdot (01)^* 0(0+1)$$

## Final Result

$$\text{Regular expression} = (0 + 1)^* (01)^* 0 (0 + 1)$$

General procedure demonstrated: (1) list all incoming transitions of every state as a right-linear equation, (2) repeatedly apply Arden's theorem starting from equations with no self-loop, (3) substitute solved variables into later equations, (4) finally solve the equation of the final state(s) — the resulting expression is the regular expression equivalent to the automaton. (For an automaton with multiple final states, the required regex is the union of the R-expressions of all final states.)

## Question 3

Consider the grammar:

$$S \rightarrow aSc \mid T \mid U \mid \varepsilon$$

$$T \rightarrow bTc \mid \varepsilon$$

$$U \rightarrow aUb \mid \varepsilon$$

### A) Language defined by the grammar

S first applies aSc zero or more times (say n times), producing a wrapper  $a^n \dots c^n$ , and then must switch to exactly one of T, U or  $\varepsilon$  (this switch happens only once, since afterwards only T-rules or only U-rules apply).

- T-branch: T generates  $b^m c^m$ , so overall we get  $a^n b^m c^{n+m}$
- U-branch: U generates  $a^k b^k$ , so overall we get  $a^{n+k} b^k c^n$  — i.e.  $a^i b^j c^k$  with  $k = i - j$
- $\varepsilon$ -branch ( $m = 0$  or  $k = 0$  above): gives  $a^n c^n$ , which is a special case of both branches

Combining both branches, every derived string has the shape  $a^i b^j c^k$  (all a's, then all b's, then all c's), and the exponents satisfy exactly one of two arithmetic relations:

$$L(G) = \{ a^i b^j c^k \mid i, j, k \geq 0 \text{ and } (k = i + j \text{ OR } (i \geq j \text{ and } k = i - j)) \}$$

### B) Derivation tree for the word "aaabbc"

$aaabbc = a^3b^2c^1$ . Since  $k = 1 = 3 - 2 = i - j$  (not  $i + j = 5$ ), this string is generated purely by the U-branch, with  $S \rightarrow aSc$  applied once ( $n = 1$ ) and  $U \rightarrow aUb$  applied twice ( $k = 2$ , matching the 2 b's).

Leftmost derivation:

$S \Rightarrow aSc \Rightarrow aUc \Rightarrow a(aUb)c \Rightarrow aa(aUb)bc \Rightarrow aaa(\epsilon)bbc \Rightarrow aaabbc$

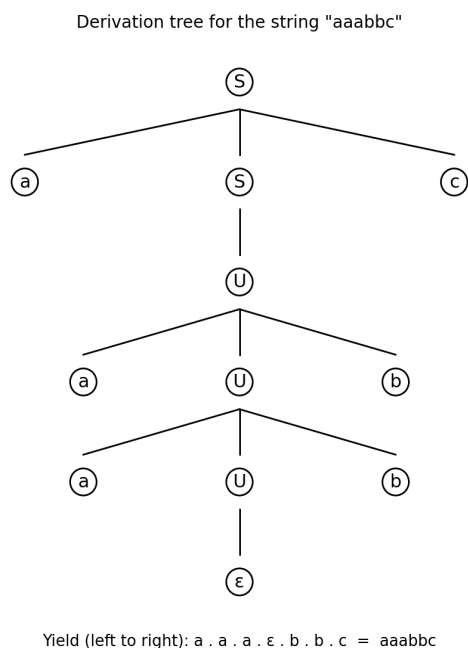


Fig. 4 — Derivation tree for "aaabbc"

C) Is the grammar ambiguous? Justify.

Yes — the grammar is ambiguous. A grammar is ambiguous if some string in its language has two distinct parse trees (equivalently, two distinct leftmost derivations).

Consider the string "ac" ( $n = 1$ , take the  $\epsilon$  case). It can be derived in two different ways, once through T and once through U:

Derivation 1 (via T):  $S \Rightarrow aSc \Rightarrow aTc \Rightarrow a(\epsilon)c = ac$

Derivation 2 (via U):  $S \Rightarrow aSc \Rightarrow aUc \Rightarrow a(\epsilon)c = ac$

These are two structurally different parse trees (one branches through  $T \rightarrow \epsilon$ , the other through  $U \rightarrow \epsilon$ ) producing the identical string "ac". Since at least one string has two distinct derivation trees, the

grammar is ambiguous. (This happens because the T-branch and U-branch overlap exactly on the string set  $\{ a^n c^n \mid n \geq 0 \}$ , which every  $\varepsilon$ -terminated derivation from either branch also produces.)

#### Question 4

i) Show that if  $L_1$  and  $L_2$  are regular, then  $L_1 \cup L_2$  is also regular

Proof (closure of regular languages under union, via product construction of DFAs):

Since  $L_1$  and  $L_2$  are regular, there exist DFAs

$M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$  accepting  $L_1$ , and

$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$  accepting  $L_2$ .

Construct a new DFA  $M = (Q, \Sigma, \delta, q_0, F)$  where:

- $Q = Q_1 \times Q_2$  (each state of  $M$  is a pair of states, one from  $M_1$  and one from  $M_2$ )
- $q_0 = (q_1, q_2)$  (start state pairs the two start states)
- $\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$  for every  $a \in \Sigma$  ( $M$  simulates  $M_1$  and  $M_2$  in parallel)
- $F = \{ (p, q) \in Q \mid p \in F_1 \text{ OR } q \in F_2 \}$  (accept if either simulated machine would accept)

By induction on the length of an input string  $w$ , one can show  $\delta^*((q_1, q_2), w) = (\delta_1^*(q_1, w), \delta_2^*(q_2, w))$ .

Hence  $w$  is accepted by  $M \Leftrightarrow \delta_1^*(q_1, w) \in F_1 \text{ or } \delta_2^*(q_2, w) \in F_2 \Leftrightarrow w \in L_1 \text{ or } w \in L_2 \Leftrightarrow w \in L_1 \cup L_2$ .

Therefore  $L(M) = L_1 \cup L_2$ , and since  $M$  is a DFA,  $L_1 \cup L_2$  is regular. ■

(Equivalently, if  $r_1$  and  $r_2$  are the regular expressions for  $L_1$  and  $L_2$ , then  $r_1 \mid r_2$  is a regular expression for  $L_1 \cup L_2$  — the class of regular expressions is closed under the union operator by definition.)

ii) Is the language  $\{ a^p \mid p \text{ is prime} \}$  regular? Justify.

No — this language is NOT regular. Proof using the Pumping Lemma for regular languages:

Assume, for contradiction, that  $L = \{ a^p \mid p \text{ is prime} \}$  is regular. Then the Pumping Lemma guarantees a pumping length  $n \geq 1$  such that every string in  $L$  of length  $\geq n$  can be split and pumped while remaining in  $L$ .



Since there are infinitely many primes, choose a prime  $p$  such that  $p \geq n + 2$ . Let  $s = a^p$ , so  $|s| = p \geq n$ .

By the Pumping Lemma,  $s$  can be written as  $s = xyz$  with:

- $|xy| \leq n$
- $|y| \geq 1$  — let  $|y| = k$ , so  $1 \leq k \leq n$
- $xy^iz \in L$  for all  $i \geq 0$

Now pump with  $i = p + 1$ :

$$|xy^{(p+1)}z| = |xyz| + p \cdot k = p + p \cdot k = p(1 + k)$$

Since  $p \geq 2$  and  $(1 + k) \geq 2$  (as  $k \geq 1$ ), the length  $p(1 + k)$  is a product of two integers each  $\geq 2$ , so it is composite — it can never be a prime number.

But the Pumping Lemma requires  $xy^{(p+1)}z \in L$ , i.e. its length must be prime. This is a contradiction.

Hence our assumption is false, and  $L = \{ a^p \mid p \text{ is prime} \}$  is NOT regular. ■